

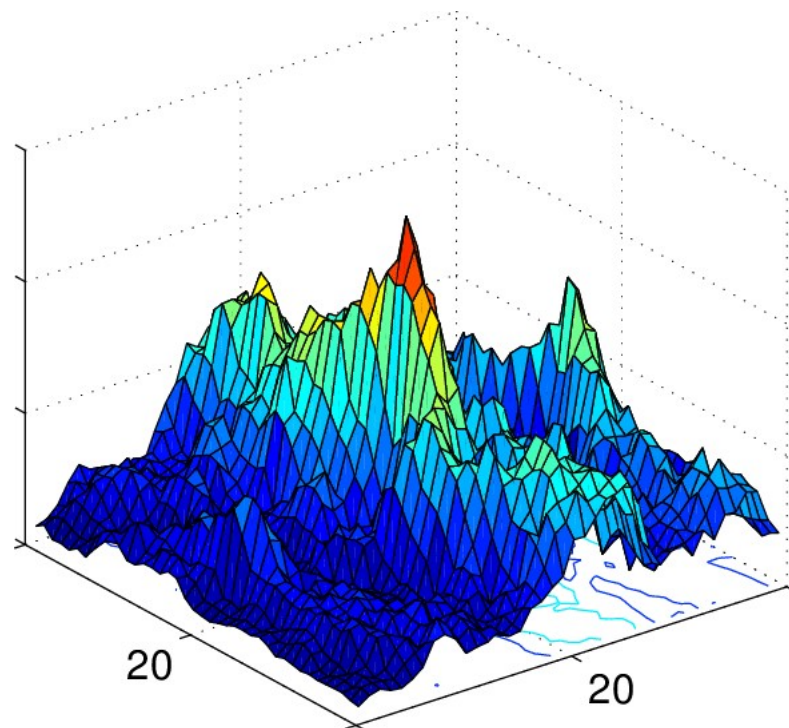
Genetic Algorithms

Management Science Principles

Heuristic Optimization Approaches

Optimization

- We introduced optimization in the context of using gradient descent to learn model parameters.
 - This can work well for most differentiable objective functions.
- If your problem is linear in nature, then a linear programming method like Simplex is sufficient.
- Some optimization problems are not so easy!
 - What to do with non-differentiable and non-linear objective functions?



Credit:

https://www.researchgate.net/figure/Non-smooth-objective-function-net-present-value-which-corresponds-to-an-optimal-well_fig2_241790004

Evolutionary Algorithms and Heuristic Methods

- Many optimization methods exist for all kinds of different problems.
 - Depends on your objectives, constraints, and other system characteristics.
 - You can find a collection of different methods with more information about each in [Wikipedia's list of optimization techniques](#).
 - We've seen purely algorithmic like Simplex, iterative like Gradient Descent, and now we'll look at heuristic approaches with an evolutionary algorithm.
- Evolutionary algorithms are a large set of optimization techniques which use concepts like reproduction, mutation, and selection to find an optimal solution.
 - In this case, our heuristics are the way we define mutations, fitness, etc.
 - This is technically a heuristic approach because it is not based on a deterministic algorithm (e.g., simplex) or defined mathematical expression (e.g., gradient of a function).
 - It is a stochastic or random process simulating possible solutions, as in [Monte Carlo methods](#).

Genetic Algorithms

Genetic Algorithms – Creating a Genetic Representation

- Genetic Algorithms are likely the best-known example of an evolutionary algorithm.
- These algorithms work by encoding a problem with a genetic representation (constructing the genome) and simulating generations as guided by a fitness function.

$$f(x) = x_1 + x_2^3 + \frac{\tan(x_3^{-5})}{|1 - x_4|}$$



Random initial values,
encoded in binary

$$x_1 = 28 = [1 \ 1 \ 1 \ 0 \ 0]$$

$$x_2 = 9 = [0 \ 1 \ 0 \ 0 \ 1]$$

$$x_3 = 16 = [1 \ 0 \ 0 \ 0 \ 0]$$

$$x_4 = 3 = [0 \ 0 \ 0 \ 1 \ 1]$$



Example gene or chromosome

[1 1 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1 1]

Initial Gene Pool and Fitness

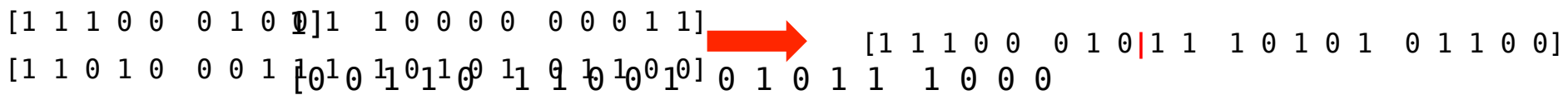
- Given the genetic representation, we can randomly generate an initial gene pool.
- Then we can evaluate this population based on a **fitness** function.
 - Fitness depends on the problem.
 - For instance, if using a genetic algorithm to build a car, you might use distance before crashing:
https://rednuht.org/genetic_cars_2/
 - In our toy problem, let's say fitness is how close the parameters are to $f(x)=1000$.

[1 1 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1 1]		$f(28,9,16,3) \cong 757$	243
[1 0 1 1 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0 1]		$f(27,6,23,9) \cong 244$	756
[1 0 1 1 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0 1]		$f(12,26,5,30) \cong 17588$	16588
[0 1 1 0 0 1 1 0 1 0 0 0 1 0 1 1 1 1 1 0]		$f(19,12,5,25) \cong 1747$	747
[0 1 1 0 0 1 1 0 1 0 0 0 1 0 1 1 1 1 1 0]		$f(6,25,11,17) \cong 15631$	14631
[1 0 0 1 1 1 1 0 0 0 0 1 1 0 1 0 1 0 0 0]		$f(26,7,21,12) \cong 370$	630

Reproduction / Crossover

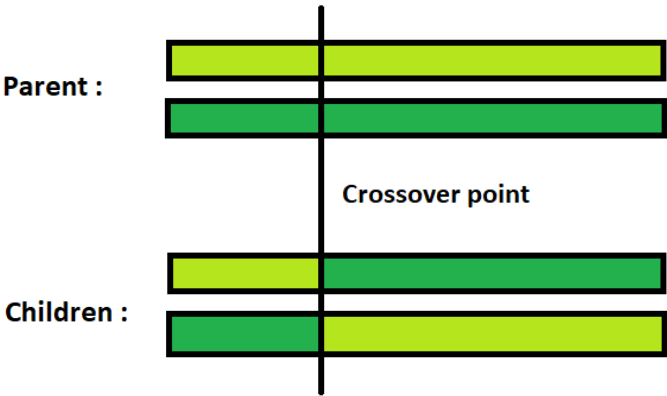
- Using a selection method, we then crossover two genes to produce a new one as a form of reproduction.
 - The selection method can be rank-based, random sampling, or other.

[1 1 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 1 1]	243
[1 1 0 1 0 0 0 1 1 1 1 0 1 0 1 0 1 1 0 0]	630
[1 0 0 1 1 1 1 0 0 0 0 0 1 1 0 1 0 1 0 0 1]	747
[1 0 1 1 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0 0]	756
[1 1 1 0 0 0 1 0 0 0 0 0 0 0 0 1 1 1]	14631
[1 1 0 1 0 0 0 1 0 1 0 1 0 1 0 0 0 0]	16588



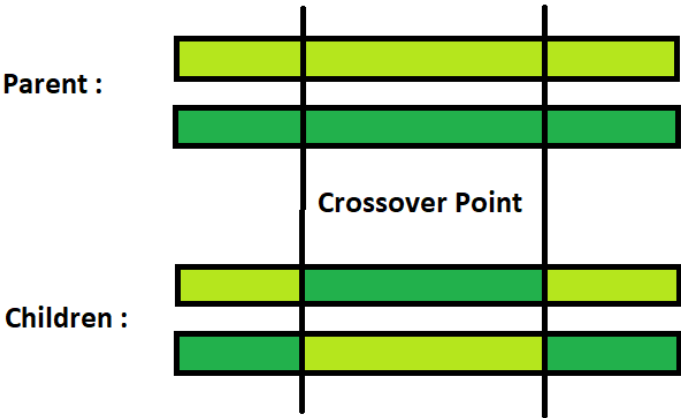
Example: From the top two parents, we could cross them over to create a new child.
This is a single-point rank-based crossover.

Crossover Points



Chromosome1	11011 00100110110
Chromosome2	11011 11000011110
Offspring1	11011 11000011110
Offspring2	11011 00100110110

Single Point Crossover



Chromosome1	11011 00100 110110
Chromosome2	10101 11000 011110
Offspring1	11011 11000 110110
Offspring2	10101 00100 011110

Two Point Crossover

Random Mutation

- Some top performers from the previous generation may be carried over directly.
 - This is called “elitism”.
- Others will be the result of a set of selections and crossovers.
- Finally, a small random chance exists of applying a mutation (bit flip) to a gene.
 - Usually only around 1% are mutated.

[1 1 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1	243
1]	359
[1 1 1 0 0 0 1 0 1 1 1 0 1 0 1 0 1 1 0	918
0]	756
[1 1 1 0 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0	630
1]	914
[1 1 0 1 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0	

Mutation Example – Initial Gene

[1 1 1 0 0 0 1 0 1 1 1 0 1 0 1 0 1 1 0
0]



X1 = 28



X2 = 11



X3 = 21



X4 = 12

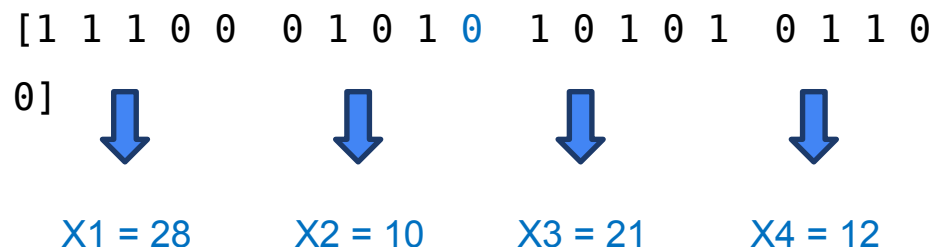
$$f(\mathbf{x}) = x_1 + x_2^3 + \frac{\tan(x_3^{-5})}{|1 - x_4|}$$

$$28 + 11^3 + \frac{\tan(21^{-5})}{|1 - 12|}$$

1359

$$\text{Fitness} = |f(\mathbf{x}) - 1000| = 359$$

Mutation Example – Mutated Gene



$$f(\mathbf{x}) = x_1 + x_2^3 + \frac{\tan(x_3^{-5})}{|1 - x_4|}$$

$$28 + 10^3 + \frac{\tan(21^{-5})}{|1 - 12|}$$

$$1028$$

$$\text{Fitness} = |f(\mathbf{x}) - 1028| = 28$$

Ranking the Next Generation

- The next generation consists of a new gene pool, each with their own fitness.
- We can rank again on fitness to prepare for the next iteration of reproduction and mutation.

[1 1 1 0 0 0 1 0 1 0 1 0 1 0 1 1 0 0]	28
[1 1 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1 1]	243
[1 1 1 0 0 1 1 0 1 0 0 0 1 0 1 0 1 0 0 1]	630
[1 1 0 1 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1]	756
[1 1 0 1 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1]	914
[1 1 0 1 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1]	918

Pruning

- Additionally, we can clean up the gene pool with **pruning**.
- Pruning consists of removing weak genes in an effort to emphasize stronger ones.
- For example, we could prune the bottom 50% from our gene pool, based on fitness, and retain only the top 50% for reproduction:

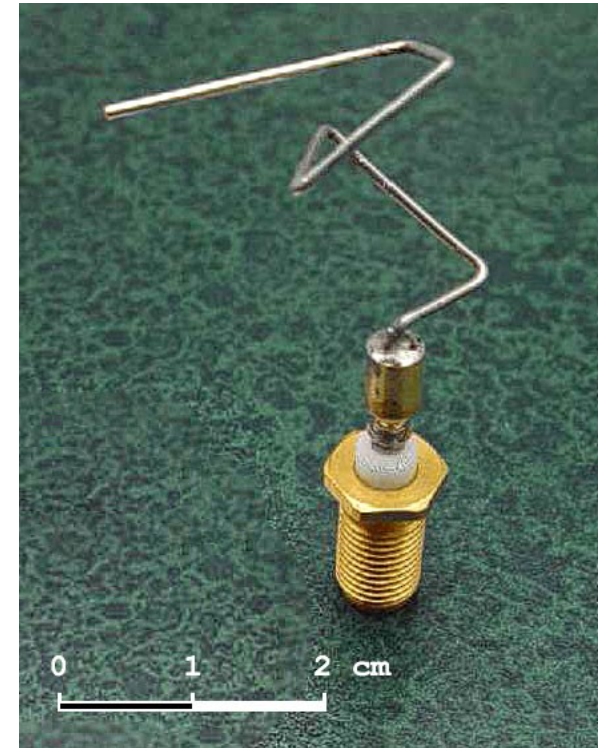
[1 1 1 0 0 0 1 0 1 0 1 0 1 0 1 1 0	28
0]	243
[1 1 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 1	630
[1]	756
[0 1 1 0 0 1 1 0 0 1 0 1 1 1 0 0 0	914
1]	918

PRUNED

Genetic Algorithm Python Example

Genetic Algorithm Summary

- Encode problem in genetic representation
 - We used binary in our example, but it could just be a list of numbers!
- Define fitness function
- Randomize first generation gene pool
- While not converged
 - Reproduce (select and crossover)
 - Mutate
 - Calculate fitness
 - Rank and prune
 - If using ranking and/or pruning vs. pure random



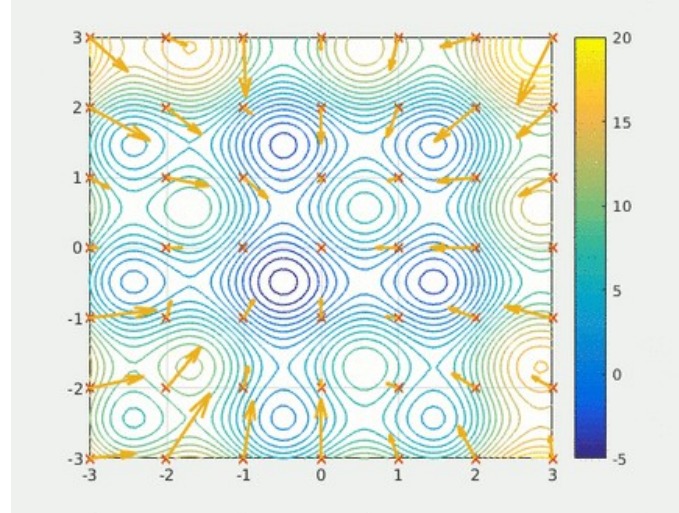
Credit: https://en.wikipedia.org/wiki/Evolved_antenna

Additional Comments

- Genetic algorithms are easy to implement if you can create a sufficient encoding and fitness function!
- You're essentially just running a simulation to try guesses.
 - What makes this better than pure randomness? The fitness and selection functions!
- There is no guarantee of convergence to an optimal final solution.
- Some visualizations:
 - [Genetic Car Racing](#)
 - <https://www.geneticalgorithms.online>
 - [Succinct video description of knapsack problem](#)

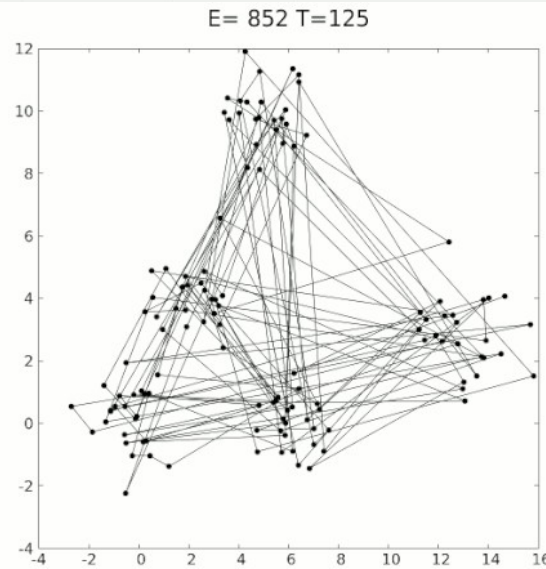
Other Optimizers

- There are many other methods that may be the best option, depending on the problem.
- Particle swarm optimization uses a swarm of particles on the surface of the objective function, with a degree of random walk, to work to a solution.
 - Related: [Swarm intelligence](https://en.wikipedia.org/wiki/Swarm_intelligence)
- Simulated annealing moves around neighboring states as a global “temperature” cools to increasingly restrict large changes.



Credit:

https://en.wikipedia.org/wiki/Particle_swarm_optimization



Credit:

https://en.wikipedia.org/wiki/Simulated_annealing